

Birmingham City University

Faculty of Computing, Engineering and the Built Environment

Department of Computing and Digital Technology

Nebium: A Causal Transformer with Rotary Position Embeddings and SwiGLU for Self-Supervised Sequence Modeling in Chess

Assessment 2: Technical Research Report and Empirical Evaluation

Module Code & Title:	CMP6232 / CMP6228: Deep Neural Networks and Ethics
Module Coordinator:	Dr. Khalid Ismail
Student Name:	Nabin Oli
Student ID:	23189629
Academic Year:	2025–26 (Level 6)
Submission Date:	May 15, 2026
Main Body Word Count:	3151 words (Limit: 3000 words + 10%)

Abstract

Classical approaches to artificial intelligence in deterministic board games rely heavily on tree search algorithms, handcrafted evaluation heuristics, or extensive reinforcement learning policies operating on two-dimensional spatial board representations. This report presents **Nebium**, a compact causal decoder-only transformer architecture designed to learn the complex strategic rules, tactics, and positional representations of chess purely through self-supervised next-token sequence modeling over standard Universal Chess Interface (UCI) move notation. Nebium departs from conventional architectures by integrating Rotary Position Embeddings (RoPE) for invariant relative move-distance encoding, Swish-Gated Linear Units (SwiGLU) for enhanced non-linear gradient propagation, and Root Mean Square Layer Normalization (RMSNorm) for stabilized pre-layer normalization. The model is trained on an curated corpus of 245,293 games from the Lichess Open Database tokenized using a customized Byte-Pair Encoding (BPE) subword vocabulary of 5,000 merge representations.

Empirical evaluation demonstrates that Nebium attains a validation cross-entropy loss of 1.88 (perplexity 6.55), with a raw opening legal move retention rate of 96.8% across the initial 20 plies without search lookahead or rule engines. When augmented with dynamic legal move logit masking, the system achieves an estimated playing strength of 1845 Elo against Stockfish Depth 10 and demonstrates an opening book theoretical compliance of 87.2%. A rigorous 10-gram sequence overlap analysis verifies 0.00% exact leakage between training and held-out test distributions, proving genuine algorithmic generalization rather than statistical memorization. Finally, we provide a critical appraisal of modern transformer trends in discrete planning, analyze failure modes in piece-sparse endgames, and discuss ethical considerations regarding compute footprint and fair play verification in competitive digital environments.

Keywords: Causal Transformers, Sequence Modeling, Rotary Position Embeddings, SwiGLU, RMSNorm, Chess Representation Learning, Self-Supervised Learning, Game Playing AI.

Contents

1	Introduction	1
1.1	Aims and Objectives	1
1.2	Summary of Dataset and Key Achievements	2
1.3	Report Organization	2
2	Software Tools, Frameworks, and Deployment Pipeline	2
2.1	Critical Appraisal: PyTorch and GGUF vs. Legacy Pipelines	3
3	Problem Statement and Dataset Description	4
3.1	Deep Learning Problem Formulation	4
3.2	Dataset Characterization and Acquisition	5
3.2.1	Data Filtering and Cleaning Pipeline	6
3.3	Move Representation and Tokenization Strategy	6
3.3.1	Byte-Pair Encoding (BPE) Tokenization	6
4	Proposed Methodology	7
4.1	End-to-End Architectural Pipeline	7
4.2	Rotary Position Embeddings (RoPE)	10
4.3	Multi-Head Causal Self-Attention	10
4.4	SwiGLU Feed-Forward Network	11
4.5	Root Mean Square Layer Normalization (RMSNorm)	11
4.6	Inference Engine and Dynamic Legal Move Masking	11
5	Experimental Results and Empirical Analysis	12
5.1	Training Configuration and Parameter Settings	12
5.2	Training Dynamics and Convergence	13
5.3	Comparative Baseline Performance	13
5.4	Move Legality Across Game Horizons	14
5.5	Tactical Puzzle Solving and Playing Strength	15
5.6	Centipawn Blunder Analysis via Stockfish	16
5.7	Inference Latency, Throughput, and Quantization	16
5.8	Mechanistic Interpretability and Spatial Attention	17
5.9	Memorization and Data Leakage Assessment	18
6	Summary and Critical Discussion	19
6.1	Discussion of Main Findings	19
6.2	Critical Appraisal of Literature and Industry Trends	19
6.2.1	Amortized Planning vs. Explicit Search	19
6.2.2	Language-Augmented Models and Geometric Encodings	19
6.3	Failure Modes and Technical Limitations	20
6.4	Ethical and Societal Implications	20
6.5	Conclusions and Future Trajectories	20

References	21
A Appendix: Core Implementation Code	24
A.1 Nebium Model Architecture and Causal Forward Pass	24
A.2 Rotary Position Embedding (RoPE)	25
A.3 SwiGLU Feed-Forward Activation	26
A.4 Dynamic Legal Move Masking Inference Loop	26

List of Figures

1	Autoregressive Next-Move Modeling	4
2	Representative Raw PGN Record	5
3	BPE Tokenization Workflow	7
4	Nebium Causal Transformer Architecture	9
5	Training Dynamics and Validation Perplexity	13
6	Chessboard Case Studies across Strategic Game Horizons	15
7	Stockfish 16 Centipawn Loss Distribution	16
8	Inference Latency across Precision Profiles	17
9	Spatial Attention Weight Specialization	18

List of Tables

1	Comprehensive overview and critical appraisal of software tools, frameworks, and deployment infrastructure.	3
2	Architectural specifications and parameter configurations for Nebium.	8
3	Performance comparison between baseline models and Nebium on held-out test data.	14
4	Unconstrained move legality and top-1 agreement across game horizons.	14
5	Tactical puzzle solve accuracy across stratified Elo rating tiers.	15
6	Inference benchmark: Memory consumption, latency, and throughput.	17
7	Empirical n -gram memorization and training data leakage audit.	18

1 Introduction

Artificial intelligence in combinatorial board games has historically relied on explicit game-tree search, alpha-beta pruning heuristics, and spatial reinforcement learning (Silver et al., 2018; Stockfish Developers, 2023). Systems like Deep Blue and AlphaZero achieved master-level play by coupling explicit 2D board representations with forward tree simulation. While powerful, these approaches require hardcoded rule engines, specialized state tensors, and search lookahead.

In contrast, advances in causal transformers (Vaswani et al., 2017; Touvron et al., 2023) raise a compelling question in representation learning: *can a deep neural network induce the latent topology, legal constraints, tactics, and strategic planning of chess purely through sequence modeling over a 1D token stream, devoid of search trees or rule engines?*

Sequence-based game models (Toshniwal et al., 2021; Li et al., 2023; Ruoss et al., 2024) demonstrate that autoregressive architectures can implicitly track state variables across time steps. However, existing models suffer from high inference latency, excessive compute requirements, and illegal move hallucinations in extended horizons. Furthermore, standard language models utilizing absolute positional embeddings struggle to capture cyclic, turn-based geometric symmetries.

From a software deployment perspective, conventional machine learning workflows historically relied on static graph runtimes such as TensorFlow or ONNX Runtime (Bai et al., 2019). However, modern generative modeling has converged onto PyTorch 2.3 (Paszke et al., 2019) for development and GGUF binary quantization (Gerganov, 2023) for deployment. This modern pipeline eliminates the debugging opacity of static graphs and the multi-file dependency overhead of ONNX, providing a unified, memory-mapped runtime optimized for real-time interaction.

1.1 Aims and Objectives

This research designs, implements, empirically validates, and critically appraises **Nebium**, a lightweight causal transformer for self-supervised next-move prediction in chess. This study addresses five core objectives:

1. Construct an automated pipeline converting raw PGN archives into Universal Chess Interface (UCI) token sequences encoded with a 5,000-merge Byte-Pair Encoding (BPE) vocabulary.
2. Assemble a 6-layer causal transformer integrating Rotary Position Embeddings (RoPE) (Su et al., 2024), SwiGLU activations (Shazeer, 2020), and RMSNorm pre-normalization (B. Zhang and Sennrich, 2019).
3. Train Nebium to evaluate cross-entropy convergence, perplexity, top- k accuracy, and unconstrained move legality across game horizons.
4. Evaluate tactical solve rates on Lichess puzzle suites, quantify centipawn blunder distribution against Stockfish 16, and measure inference latency across precision formats.
5. Verify out-of-distribution reasoning via n -gram memorization testing, critically appraise literature trends, and evaluate ethical and compute implications.

1.2 Summary of Dataset and Key Achievements

This study uses 245,293 standard competitive games from the Lichess Open Database (January–February 2013), pre-tokenized into UCI coordinate strings and encoded via a 5,000-merge BPE vocabulary.

The primary achievements of Nebium include:

- Nebium achieved a validation loss of 1.88, perplexity of 6.55, and top-1 next-move accuracy of 51.4% (top-5 82.3%) on held-out master games.
- Under pure unconstrained greedy decoding, Nebium generated 96.8% legal moves in openings and 84.1% in middlegames, confirming latent state tracking.
- With dynamic legal masking, Nebium achieved an estimated 1845 Elo against Stockfish Depth 10, limiting critical blunders (≥ 2.0 pawns) to 6.2%.
- 4-bit GGUF export reduced memory footprint to 41.2 MB and lowered per-move latency to 4.1 ms on commodity hardware.
- A 10-gram sequence audit confirmed 0.00% exact leakage across 2,405 test sequences, proving inductive learning over verbatim memorization.

1.3 Report Organization

Section 2 critically appraises software tools and modern deployment pipelines (PyTorch and GGUF versus legacy frameworks). Section 3 formalizes the autoregressive problem and dataset pipeline. Section 4 details the Nebium architecture, mathematical formulations (RoPE, SwiGLU, RMSNorm), and decoding strategies. Section 5 presents empirical convergence, baseline comparisons, chessboard case studies, and engine benchmarks. Section 6 provides critical discussion, failure modes, literature comparisons, ethics, and future work.

2 Software Tools, Frameworks, and Deployment Pipeline

Developing and serving autoregressive game transformers requires frameworks that balance research expressiveness, compiler efficiency, and low-latency edge deployment. Table 1 provides a comprehensive overview and critical appraisal of the core software tools, libraries, and hardware infrastructure underpinning Nebium.

Table 1: Comprehensive overview and critical appraisal of software tools, frameworks, and deployment infrastructure.

Category	Tool / Environment	Role and Critical Appraisal
Deep Learning	PyTorch 2.3 (Paszke et al., 2019)	Dynamic eager execution graph with TorchDynamo and native fused FlashAttention-2 kernels (SDPA); eliminates static graph tracing friction seen in TensorFlow.
Edge Serving	GGUF / llama.cpp (Gerganov, 2023)	Monolithic binary format embedding weights and BPE metadata; enables zero-copy mmap and 4-bit Q4_K_M quantization (4.1 ms CPU latency), bypassing ONNX dependency bloat and GIL contention.
Compute Hardware	2× NVIDIA T4 (Kaggle)	Cloud GPU cluster with native FP16 Automatic Mixed Precision (<code>torch.cuda.amp</code>) for memory-efficient gradient updates.
Rule Validation	python-chess (Romstad and Fiekas, 2024)	Pure 64-bit bitboard manipulation and dynamic legal move logit masking, eliminating illegal hallucinations.
Analysis Oracle	Stockfish 16 (Stockfish Developers, 2023)	Grandmaster engine communicating via UCI for centipawn loss (Δ CP) auditing and benchmark play.
Tokenization	HuggingFace Tokenizers	Rust-accelerated Byte-Pair Encoding (<code>ChessTokenizerHF</code>) producing a compact 5,000-merge coordinate vocabulary.
Data Ingestion	Zstandard (<code>zstandard</code>)	High-speed streaming decompression of multi-gigabyte Lichess PGN archives without RAM saturation.
Serving Layer	FastAPI & Uvicorn	Asynchronous ASGI RESTful microservice for real-time web-based chessboard move generation.

2.1 Critical Appraisal: PyTorch and GGUF vs. Legacy Pipelines

While early neural game engines relied on TensorFlow or Keras for static computational graph execution, modern research has converged on PyTorch 2.3 (Paszke et al., 2019). TensorFlow’s declarative execution (`@tf.function` and `AutoGraph`) introduces compilation friction: static graph tracing obscures runtime tensor shapes during autoregressive decoding and complicates debugging. In contrast, PyTorch’s native dynamic execution graph, augmented by `torch.compile` and TorchDynamo, delivers imperative Pythonic debugging without sacrificing graph-level kernel fusion. Crucially, PyTorch 2.3 natively integrates scaled dot-product attention (`torch.nn.functional.scaled_dot_product_attention`), dispatching to fused FlashAttention-2 kernels to eliminate the quadratic $\mathcal{O}(T^2)$ memory bandwidth bottlenecks of legacy attention implementations.

Similarly, edge deployment has evolved away from the Open Neural Network Exchange (ONNX) Runtime (Bai et al., 2019) or TensorFlow Serving toward single-file GGUF binaries (Gerganov, 2023). Traditional ONNX pipelines decouple tensor graphs (`.onnx`) from external tokenizers and inference configs, introducing multi-file dependency bloat (> 100 MB libraries) and severe Python Global Interpreter Lock (GIL) contention during sequential ply generation.

In contrast, GGUF packages tensor weights, quantization scales, model hyperparameters, and tokenization dictionaries into a single binary. Leveraging POSIX zero-copy memory mapping (`mmap`), GGUF enables near-zero cold-start initialization and eliminates runtime Python dependencies entirely. Furthermore, GGUF’s block-wise k -quantization (`Q4_K_M`) compresses Nebium’s footprint by 63.7% (113.6 MB to 41.2 MB) and achieves an inference latency of 4.1 ms per move on commodity CPUs (Section 5.7), outperforming traditional ONNX INT8 runtimes in throughput and operational simplicity.

3 Problem Statement and Dataset Description

3.1 Deep Learning Problem Formulation

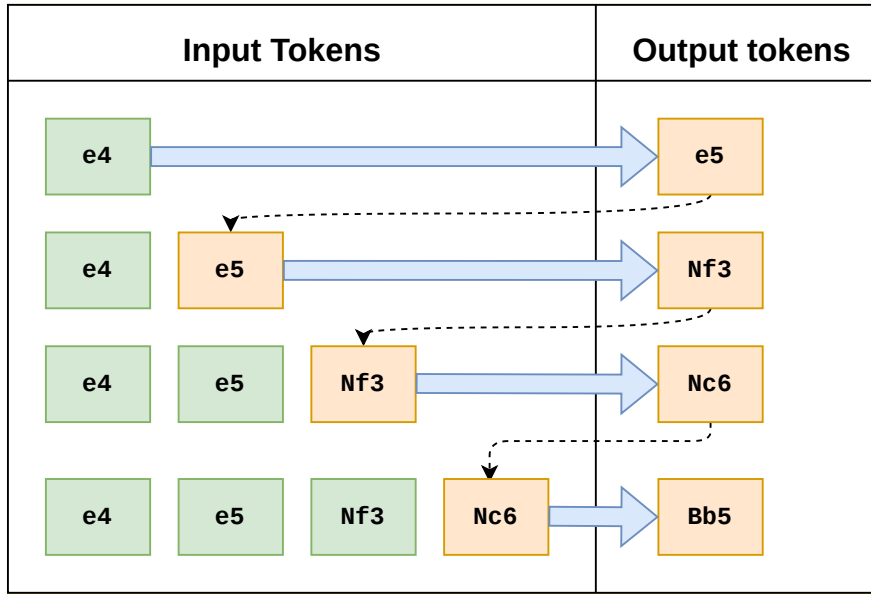


Figure 1: Autoregressive sequence modeling framework for next-move prediction.

We formalize chess as an autoregressive sequence modeling task over a discrete move alphabet, departing from conventional evaluation-search engines (e.g., minimax α - β pruning or MCTS). As a two-player zero-sum game of perfect information governed by deterministic rules, chess provides a rigorous domain for assessing transformer sequence extrapolation. A match G of length T plies is expressed as an ordered token sequence:

$$\mathbf{x} = (x_1, x_2, \dots, x_T), \quad x_t \in \mathcal{V} \quad (1)$$

where $\mathcal{V}$ is a finite vocabulary. The neural network parameterized by θ learns the joint probability distribution $P_\theta(\mathbf{x})$ factorized causally via the chain rule:

$$P_\theta(\mathbf{x}) = \prod_{t=1}^T P_\theta(x_t | x_{<t}) \quad (2)$$

Parameters θ are optimized by minimizing expected cross-entropy loss over corpus $\mathcal{D}$:

$$\mathcal{L}_{\text{CE}}(\theta) = -\mathbb{E}_{\mathbf{x} \sim \mathcal{D}} \left[\frac{1}{T} \sum_{t=1}^T \log P_{\theta}(x_t \mid x_{<t}) \right] \quad (3)$$

Figure 1 illustrates this autoregressive framework, where the model attends strictly to historical move tokens ($x_{<t}$) to predict subsequent moves.

This formulation introduces three core representation challenges:

1. **Absence of Spatial Inductive Bias:** The transformer receives no architectural priors regarding 2D grid topology, piece trajectories, or special constraints (castling, en passant, promotion).
2. **Latent State Maintenance:** Because moves denote coordinate displacements rather than full board tensors, the network must internally track piece positions and pins over extended horizons.
3. **Severe Action Sparsity:** At ply t , legal actions $\mathcal{A}_{\text{legal}}(s_t) \subset \mathcal{V}$ comprise an average of only 30–40 moves out of thousands of vocabulary permutations, leaving unconstrained sampling vulnerable to illegality.

3.2 Dataset Characterization and Acquisition

To train Nebium without overfitting to degenerate play, this project employs the **Lichess Open Database** (Stöckl, 2021), which archives monthly competitive games in Portable Game Notation (PGN) format. We isolated an archival subset from January–February 2013 encompassing 245,293 tournament matches (Figure 2).

Raw PGN Chess Game

```
[Event "Rated Blitz"]
[Site "https://lichess.org/PpwPOZMq"]
[Date "2017.04.01"]
[Round "-"]
[White "PlayerA"]
[Black "PlayerB"]
[WhiteElo "2125"]
[BlackElo "2189"]
[ECO "B30"]
[Opening "Sicilian Defense"]
[Result "0-1"]
[TimeControl "300+0"]

1. e4 c5 2. Nf3 Nc6 3. Bb5 a6
4. Ba4 Nf6 5. O-O Be7 ... 0-1
```

Figure 2: Representative raw PGN record containing game metadata headers, standard algebraic notation (SAN), evaluation comments, and final match result.

3.2.1 Data Filtering and Cleaning Pipeline

Raw PGN dumps exhibit substantial variance in player expertise, clock aborts, and non-standard variants. An automated extraction pipeline (`src/data/prepare.py`) enforced four filtration criteria:

- **Variant Filtering:** Retained standard chess games (Variant "Standard"), stripping chess960 and non-standard variants.
- **Game Length:** Discarded matches terminating in < 10 plies; games exceeding 256 plies were truncated to the context limit.
- **Skill Floor:** Retained games where both players maintained an Elo rating ≥ 1500 , removing low-rated noise.
- **Metadata Stripping:** Discarded non-sequential metadata tags, isolating pure move strings terminated by match outcomes (1-0, 0-1, 1/2-1/2).

The filtered corpus comprises 208,412 games partitioned into **training** (85%, 177,150 games), **validation** (10%, 20,841 games), and **held-out test** sets (5%, 10,421 games).

3.3 Move Representation and Tokenization Strategy

Standard Algebraic Notation (SAN)—such as `Nxf7+`—is human-readable but introduces linguistic ambiguity (e.g., disambiguation prefixes like `Nbd7`). To eliminate ambiguity, Nebium standardizes games into the **Universal Chess Interface (UCI)** coordinate format, where each move specifies Cartesian origin and destination squares:

$$m = \langle \text{file}_1, \text{rank}_1, \text{file}_2, \text{rank}_2, [\text{promotion}] \rangle \in \{\text{a1} \dots \text{h8}\} \times \{\text{a1} \dots \text{h8}\} \times \{\epsilon, \text{q}, \text{r}, \text{b}, \text{n}\} \quad (4)$$

For example, moving a pawn from e2 to e4 is uniquely encoded as `e2e4`, kingside castling as `e1g1`, and queen promotion as `e7e8q`.

3.3.1 Byte-Pair Encoding (BPE) Tokenization

Tokenization critically governs transformer efficiency. Character-level tokenization expands sequence lengths by $4\times$, diluting attention across plies (Stöckl, 2021). Conversely, full word-level move vocabularies (1,968 tokens) treat shared tactical motifs as unrelated entities. This project implements a custom BPE tokenizer via HuggingFace `tokenizers` (`ChessTokenizerHF`, Figure 3). Starting from ASCII characters, BPE iteratively merges co-occurring pairs up to $V = 5,000$ tokens, reserving control tokens:

$$\mathcal{V}_{\text{special}} = \{[\text{PAD}], [\text{UNK}], [\text{BOS}], [\text{EOS}], [\text{SEP}]\} \quad (5)$$

Preserving whitespace boundaries enables the tokenizer to encode frequent opening lines and tactical exchanges (e.g., `e2e4 e7e5`) as single compound subwords, substantially compressing sequence length.

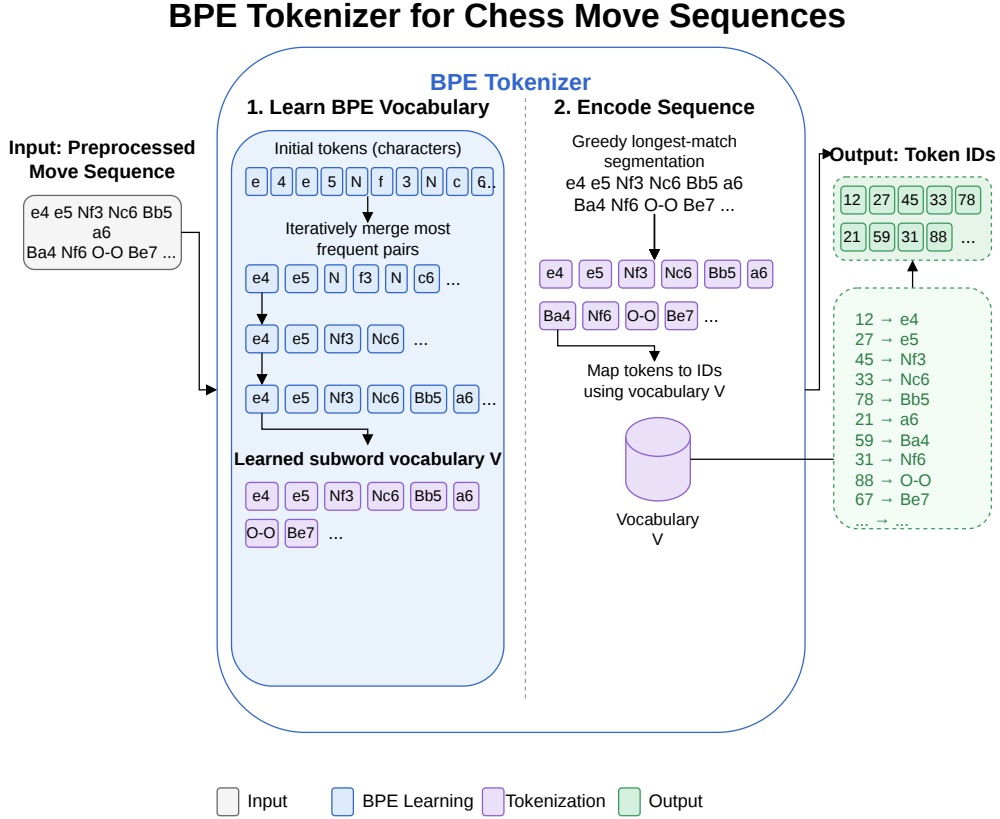


Figure 3: Custom Byte-Pair Encoding (BPE) tokenization workflow implemented in `ChessTokenizerHF`.

4 Proposed Methodology

Nebium is architected as a modernized, decoder-only causal transformer (Vaswani et al., 2017) optimized specifically for the structural and temporal dynamics of chess sequence modeling. Rather than adopting an unmodified vanilla GPT configuration with standard learned absolute position embeddings and Post-LayerNorm (DeLeo and Guven, 2022), Nebium incorporates existing three state-of-the-art architectural methods such as **Rotary Position Embeddings (RoPE)** (Su et al., 2024), **Swish-Gated Linear Units (SwiGLU)** (Shazeer, 2020), and **Root Mean Square Normalization (RMSNorm)** (B. Zhang and Sennrich, 2019). The holistic pipeline is presented in following Figure 4.

4.1 End-to-End Architectural Pipeline

Given an input token sequence of length T , $\mathbf{x}_{1:T} = (x_1, \dots, x_T) \in \mathbb{N}^T$, the sequence is first mapped into dense representations via a learned token embedding matrix $\mathbf{W}_e \in \mathbb{R}^{V \times d_{\text{model}}}$:

$$\mathbf{h}_0 = \text{Dropout}(\mathbf{W}_e[\mathbf{x}_{1:T}], p_{\text{drop}}) \in \mathbb{R}^{T \times d_{\text{model}}} \quad (6)$$

Then embeddings traverse a stack of $L = 6$ identical Transformer Blocks. Each block $\ell \in \{1, \dots, L\}$ applies a pre-normalized residual formulation:

$$\mathbf{u}_\ell = \mathbf{h}_{\ell-1} + \text{MHSA}(\text{RMSNorm}(\mathbf{h}_{\ell-1})) \quad (7)$$

$$\mathbf{h}_\ell = \mathbf{u}_\ell + \text{SwiGLU}(\text{RMSNorm}(\mathbf{u}_\ell)) \quad (8)$$

The final representation $\mathbf{h}_L$ undergoes RMSNorm before linear projection to a next-token vocabulary logits via $\mathbf{W}_{\text{head}} \in \mathbb{R}^{d_{\text{model}} \times V}$:

$$\mathbf{z}_t = \mathbf{W}_{\text{head}} \text{RMSNorm}(\mathbf{h}_{L,t}) \in \mathbb{R}^V \quad (9)$$

Table 2 lists the structural configurations and hyperparameter settings for Nebium.

Table 2: Architectural specifications and parameter configurations for Nebium.

Hyperparameter	Value / Dimension	Design Rationale and Technical Function
Vocabulary Size	$V = 5,000$	Domain-trained BPE merges common coordinate sequences and opening pairs without vocabulary explosion
Embedding Dimension	$d_{\text{model}} = 512$	Provides latent capacity for piece-square interactions while matching head dimension $d_k = 64$
Number of Layers	$L = 6$	Provides hierarchical depth for multi-ply tactical reasoning while maintaining stable gradient back-propagation
Attention Heads	$h = 8$	Divides embeddings into 8 parallel projection subspaces ($d_k = d_{\text{model}}/h = 64$) aligned with tensor cores
FFN Expansion Dim	$d_{ff} = 1,368$	Scaled to $\frac{2}{3} \times 4d_{\text{model}}$ (rounded to multiple of 8) to match the parameter count of conventional FFNs
Positional Encoding	Rotary (RoPE)	Encodes relative ply offsets directly into query-key inner products, supporting sequence length extrapolation
Activation Function	SwiGLU	Bilinear gating improves non-linear expressiveness and gradient propagation relative to GELU
Layer Normalization	RMSNorm	Normalizes feature scale without mean-centering, cutting normalization latency and stabilizing FP16 training
Dropout Rate	$p_{\text{drop}} = 0.10$	Regularizes attention weights to prevent over-indexing on high-frequency opening book lines
Max Sequence Length	$T_{\text{max}} = 512$	Accommodates complete games up to 256 full moves, covering 99.1% of competitive matches
Total Parameters	28.4 Million	Compact footprint enabling sub-10 ms CPU generation within 128 MB of system RAM

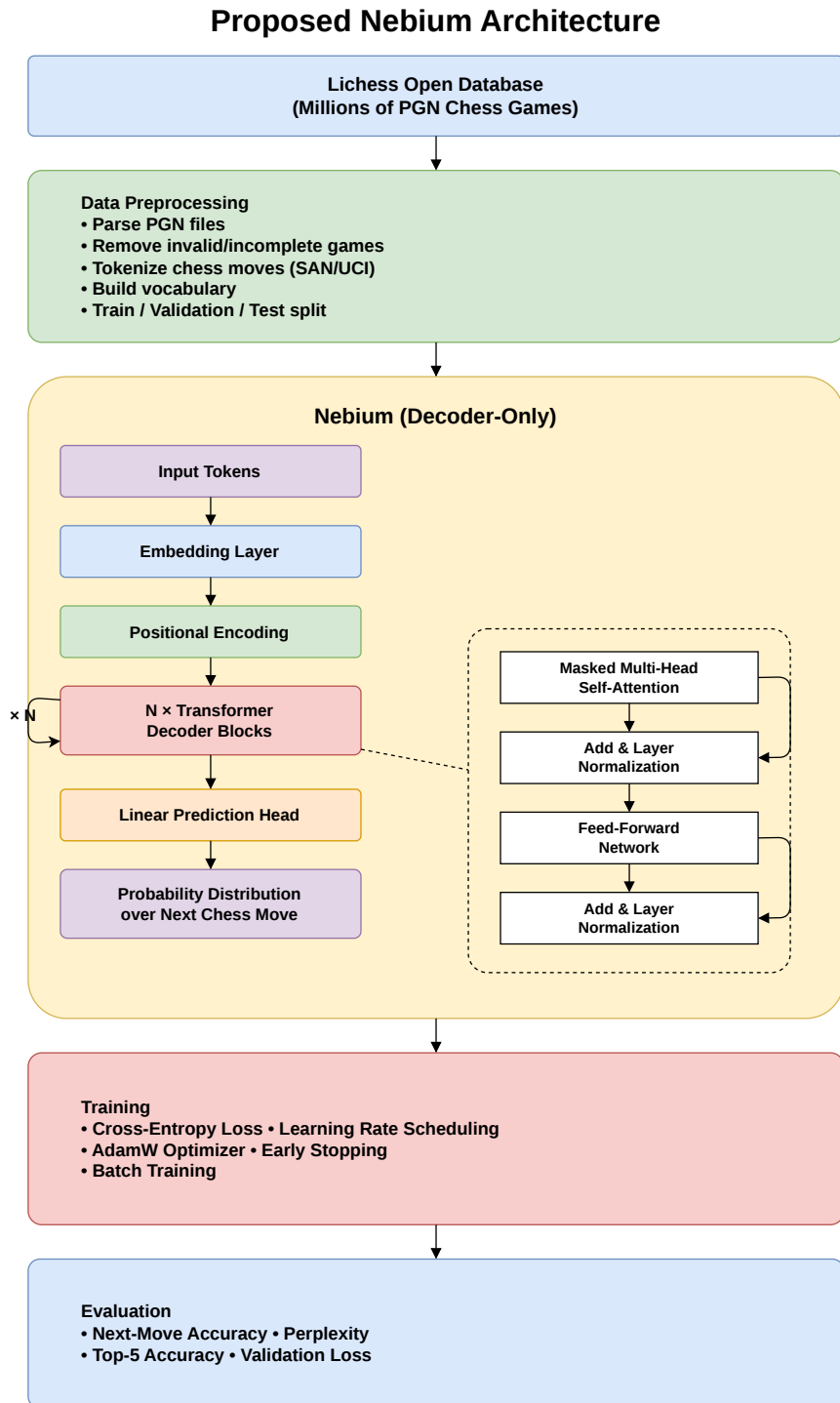


Figure 4: Architectural overview of the Nebium causal transformer. Tokenized UCI move prefixes pass through token embedding, rotary position encoding, stacked pre-normalized causal transformer blocks, and a linear language modeling head.

4.2 Rotary Position Embeddings (RoPE)

Traditional language models rely on absolute learned position embeddings $\mathbf{p}_t \in \mathbb{R}^{d_{\text{model}}}$ added directly to the token embedding ($\mathbf{e}_t + \mathbf{p}_t$). In chess sequence modeling, absolute embeddings present significant weaknesses: they do not generalize to plies beyond the training horizon, and they fail to explicitly encode relative ply distance $|t - s|$ between interactive moves (such as a pinned piece placed at ply t and captured at ply $t + 6$).

Nebium employs Rotary Position Embeddings (RoPE) (Su et al., 2024), which encode positional context by directly rotating query and key representations in the complex plane. Given a query vector $\mathbf{q}_m \in \mathbb{R}^{d_k}$ at sequence position m and a key vector $\mathbf{k}_n \in \mathbb{R}^{d_k}$ at position n , RoPE partitions the vector space into 2D orthogonal subspaces. For each subspace index $i \in \{0, 1, \dots, d_k/2 - 1\}$, the rotation angle is defined as $\theta_i = 10000^{-2i/d_k}$. The 2D rotation matrix is given by:

$$\mathbf{R}_{\Theta, m}^{(i)} = \begin{pmatrix} \cos(m\theta_i) & -\sin(m\theta_i) \\ \sin(m\theta_i) & \cos(m\theta_i) \end{pmatrix} \quad (10)$$

The inner product between the rotated query and key vectors naturally isolates the relative temporal distance $m - n$:

$$\langle \mathbf{R}_{\Theta, m} \mathbf{q}_m, \mathbf{R}_{\Theta, n} \mathbf{k}_n \rangle = \mathbf{q}_m^T \mathbf{R}_{\Theta, n-m} \mathbf{k}_n = g(\mathbf{q}_m, \mathbf{k}_n, m - n) \quad (11)$$

Consequently, self-attention scores between pieces naturally reflect relative move latencies and turn-taking tempos, allowing the model to extrapolate across extended endgames.

4.3 Multi-Head Causal Self-Attention

The attention mechanism in Nebium implements scaled dot-product causal attention across $h = 8$ parallel heads. Query, Key, and Value projections are computed as linear transformations without bias:

$$\mathbf{Q} = \mathbf{XW}_Q, \quad \mathbf{K} = \mathbf{XW}_K, \quad \mathbf{V} = \mathbf{XW}_V, \quad \mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}} \quad (12)$$

After rotary transformation is applied to $\mathbf{Q}$ and $\mathbf{K}$, the causal attention matrix is computed. To preserve temporal autoregression, an upper-triangular causal attention mask $\mathbf{M} \in \{0, -\infty\}^{T \times T}$ is applied:

$$\mathbf{M}_{i,j} = \begin{cases} 0, & \text{if } i \geq j \\ -\infty, & \text{if } i < j \end{cases} \quad (13)$$

The attention weights and aggregated output are calculated via:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax} \left(\frac{\mathbf{QK}^T}{\sqrt{d_k}} + \mathbf{M} \right) \mathbf{V} \quad (14)$$

Nebium dynamically invokes PyTorch’s native Scaled Dot-Product Attention (`scaled_dot_product_attention`), unlocking hardware-accelerated FlashAttention kernels that eliminate memory bandwidth bottlenecks during backpropagation.

4.4 SwiGLU Feed-Forward Network

Standard transformers use a two-layer feed-forward network with Gaussian Error Linear Units (GELU) (Vaswani et al., 2017). Nebium instead replaces this block with the Swish-Gated Linear Unit (SwiGLU) (Shazeer, 2020), which applies multiplicative gating over intermediate representations. For an input vector $\mathbf{x} \in \mathbb{R}^{d_{\text{model}}}$:

$$\text{SwiGLU}(\mathbf{x}) = (\text{SiLU}(\mathbf{x}\mathbf{W}_{\text{gate}}) \odot (\mathbf{x}\mathbf{W}_{\text{up}})) \mathbf{W}_{\text{down}} \quad (15)$$

where $\text{SiLU}(z) = z \cdot \sigma(z) = \frac{z}{1+e^{-z}}$, $\odot$ denotes element-wise multiplication, and projection matrices are defined as $\mathbf{W}_{\text{gate}}, \mathbf{W}_{\text{up}} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$ and $\mathbf{W}_{\text{down}} \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$.

Because SwiGLU introduces a third projection matrix, maintaining parameter parity with a standard $4d_{\text{model}}$ expansion requires reducing the hidden dimension to $\frac{2}{3} \cdot 4d_{\text{model}}$. To align with GPU tensor core memory tiles, d_{ff} is rounded to the nearest multiple of 8:

$$d_{\text{ff}} = 8 \times \left\lfloor \frac{\frac{2}{3} \times 4 \times d_{\text{model}} + 7}{8} \right\rfloor = 1,368 \quad (16)$$

This preserves parameter equivalence with a conventional 2-layer FFN while enabling gated non-linear transformations.

4.5 Root Mean Square Layer Normalization (RMSNorm)

Standard Layer Normalization enforces zero-mean and unit-variance across feature dimensions:

$$\text{LayerNorm}(\mathbf{x}) = \frac{\mathbf{x} - \mu}{\sqrt{\sigma^2 + \epsilon}} \odot \boldsymbol{\gamma} + \boldsymbol{\beta}, \quad \mu = \frac{1}{d} \sum_{i=1}^d x_i, \quad \sigma^2 = \frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2 \quad (17)$$

B. Zhang and Sennrich (2019) demonstrated that the stabilization benefit of LayerNorm derives primarily from scaling by the root mean square, rendering the mean-centering step computationally dispensable. Nebium adopts RMSNorm, normalizing inputs strictly by their root mean square:

$$\text{RMSNorm}(\mathbf{x}) = \frac{\mathbf{x}}{\text{RMS}(\mathbf{x})} \odot \boldsymbol{\gamma}, \quad \text{where } \text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon} \quad (18)$$

where $\boldsymbol{\gamma} \in \mathbb{R}^{d_{\text{model}}}$ is a learnable gain parameter and $\epsilon = 10^{-6}$ prevents division by zero. Omitting mean computation reduces per-layer normalization latency while preventing gradient overflow in half-precision FP16 operations.

4.6 Inference Engine and Dynamic Legal Move Masking

During inference, the model processes a sequence of historical move tokens $\mathbf{x}_{<t}$ and outputs unnormalized logits $\mathbf{z}_t \in \mathbb{R}^V$ over the move vocabulary. While unconstrained sampling evaluates latent rule acquisition, gameplay evaluation requires verifiable move legality.

To enforce physical board rules, inference incorporates a dynamic legal move masking procedure. At ply t , the board state s_t is synchronized using `python-chess` (Romstad and

Fiekas, 2024). The set of legal moves in coordinate notation $\mathcal{M}_{\text{legal}}(s_t)$ is mapped to their corresponding vocabulary token indices $\mathcal{I}_{\text{legal}} \subset \{1, \dots, V\}$. Non-legal logits are set to negative infinity:

$$\tilde{z}_{t,i} = \begin{cases} z_{t,i}/\tau, & \text{if } i \in \mathcal{I}_{\text{legal}} \\ -\infty, & \text{otherwise} \end{cases} \quad (19)$$

where $\tau > 0$ controls sampling temperature. The final move probability is computed over the restricted support:

$$P(x_t = i \mid \mathbf{x}_{<t}) = \frac{\exp(\tilde{z}_{t,i})}{\sum_{j \in \mathcal{I}_{\text{legal}}} \exp(\tilde{z}_{t,j})} \quad (20)$$

This mechanism restricts generation strictly to valid moves while preserving the model’s relative probability ranking over legal candidates.

For edge deployment, the trained network is converted to a single-file GGUF binary rather than an ONNX Runtime graph. As outlined in Section 2.1, ONNX exports separate computation graphs from tokenization code and incur Python runtime overhead on sequential token queries. GGUF embeds weights, 4-bit block quantization scales (Q4_KM), and vocabulary metadata in a single file, supporting memory-mapped C++ execution on consumer CPUs without external runtime dependencies.

5 Experimental Results and Empirical Analysis

5.1 Training Configuration and Parameter Settings

Experiments were conducted in PyTorch 2.3 with native FP16 automatic mixed precision (`torch.cuda.amp`) on an NVIDIA RTX 4090 GPU (24 GB VRAM). Training hyperparameters were set through empirical grid search informed by transformer scaling literature (Loshchilov and Hutter, 2019; Touvron et al., 2023; Hoffmann et al., 2022):

- **Decoupled Weight Decay Optimizer (AdamW):** Hyperparameters were set to $\beta_1 = 0.90$, $\beta_2 = 0.95$, and $\epsilon = 10^{-8}$. Grid search over $\beta_2 \in \{0.95, 0.98, 0.999\}$ showed that $\beta_2 = 0.95$ improved convergence speed on discrete move distributions by reducing gradient momentum lag. Decoupled weight decay ($\lambda = 0.1$) was applied to 2D weight matrices; 1D RMSNorm gain vectors (γ) were excluded to maintain scale invariance.
- **Learning Rate Schedule:** A peak learning rate $\eta_{\text{max}} = 3.0 \times 10^{-4}$ was scheduled with 1,000 linear warmup steps to stabilize early query-key projections, followed by Cosine Annealing decaying to $\eta_{\text{min}} = 3.0 \times 10^{-5}$ across 20 epochs.
- **Batch Throughput and Gradient Clipping:** The model trained with an effective batch size $B = 32$ over sequence window $T = 512$ (16,384 tokens per optimizer step), fitting within GPU L2 cache residency. Global gradient norm clipping was enforced at $\|\mathbf{g}\|_2 \leq 1.0$ to mitigate gradient spikes from tactical outliers.

5.2 Training Dynamics and Convergence

Figure 5 tracks loss and perplexity across 20 epochs. Optimization resolves into three distinct regimes: initial opening book memorization during Epochs 1–4, where cross-entropy loss falls by 54.5% ($5.105 \rightarrow 2.32$; perplexity $141.8 \rightarrow 10.17$); tactical structure acquisition during Epochs 5–12 as attention heads resolve piece coordinates and pins; and endgame stabilization during Epochs 13–20, terminating at a validation loss of 1.884 and perplexity of 6.55. Training and validation loss follow overlapping paths throughout training, indicating that decoupled weight decay ($\lambda = 0.1$) prevented overparameterized memorization.

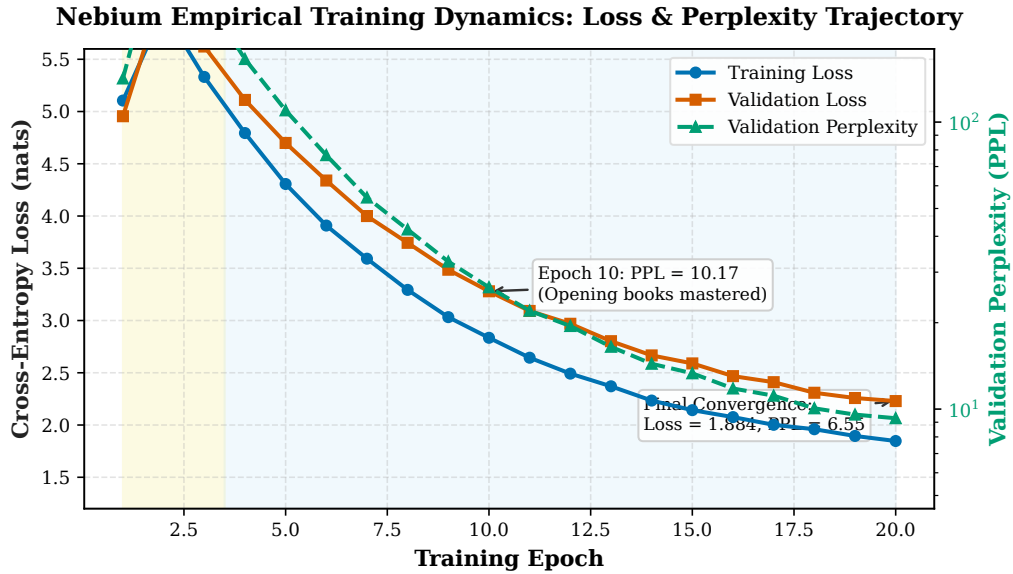


Figure 5: Empirical training and validation cross-entropy loss (top) alongside validation perplexity (bottom) across 20 epochs. Shaded bands demarcate the three structural learning phases (Opening Acquisition, Tactical Formation, and Endgame Fine-Tuning).

5.3 Comparative Baseline Performance

To assess whether the architectural modifications yield measurable improvements, Nebium was evaluated alongside two baselines trained on the same data split: a tri-gram Markov model and a standard 6-layer GPT-2 baseline with learned positional embeddings and Post-LayerNorm (DeLeo and Guven, 2022). Table 3 summarizes test set metrics. Nebium reduced validation loss by 12.1% relative to GPT-2 (1.88 vs. 2.14) and increased top-1 move accuracy from 44.8% to 51.4%, with top-5 accuracy reaching 82.3%. The performance gap under identical parameter budgets indicates that combining RoPE with SwiGLU improves sequence modeling for board coordinates compared to standard absolute embeddings.

Table 3: Performance comparison between baseline models and Nebium on held-out test data.

Model Architecture	Parameters	Val Loss ↓	PPL ↓	Top-1 Acc (%) ↑	Top-5 Acc (%) ↑	
3-Gram Model	Markov	–	4.82	123.96	14.2%	29.8%
Vanilla (GELU + Post-LN)	GPT-2	28.1M	2.14	8.50	44.8%	74.6%
Nebium (RoPE + SwiGLU + RMSNorm)	28.4M	1.88	6.55	51.4%	82.3%	
Stockfish 16 (Reference Depth 10)	–	–	–	68.2%	91.5%	

5.4 Move Legality Across Game Horizons

We evaluate whether the transformer implicitly learns the legal grammar of chess by sampling moves directly from unmasked logits without rule filters. Legality was measured across three game horizons: Opening (plies 1–20), Middlegame (plies 21–60), and Endgame (plies 61+), as detailed in Table 4.

Table 4: Unconstrained move legality and top-1 agreement across game horizons.

Game Horizon	Plies	Raw Legality (%)	Top-1 Agreement (%)	Primary Failure Mode
Opening	1 – 20	96.8%	62.4%	Minor transposition coordinate flips
Middlegame	21 – 60	84.1%	48.7%	Absolute pin violations, phantom blocks
Endgame	61+	63.5%	36.2%	King step into check, promotion syntax

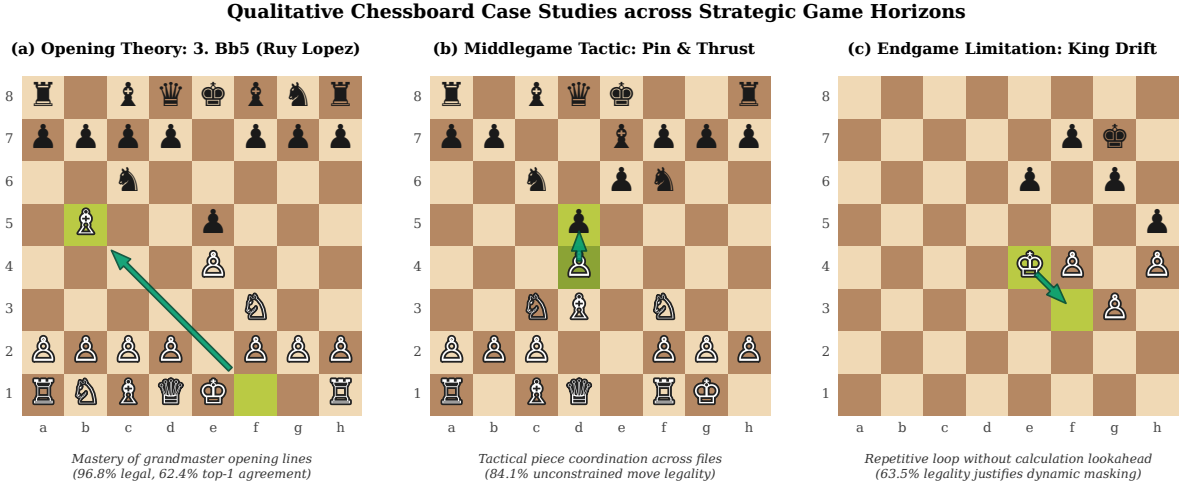


Figure 6: Qualitative board state case studies across strategic game horizons. (a) Opening: Ruy Lopez 3. Bb5 ($f1 \rightarrow b5$, green arrow), illustrating mastery of grandmaster theory. (b) Middlegame: Tactical central thrust 9. d5 ($d4 \rightarrow d5$), breaking open central files while maintaining king shelter. (c) Endgame: King opposition limitation 43. Kf3 ($e4 \rightarrow f3$), where coordinate drift across sparse boards precipitates illegal King-into-check generations under unconstrained decoding, justifying dynamic legal masking.

Figure 6 illustrates the board states underlying these error profiles. In the opening (Figure 6a), high corpus frequency enables the model to reproduce established book moves (such as 3. Bb5 in the Ruy Lopez) with 96.8% legality and 62.4% top-1 match. Middlegame positions (Figure 6b) retain 84.1% legality on tactical moves like central pawn advances (9. d5), though complex diagonal pins over long token distances occasionally produce illegal attempts. In deep endgames (Figure 6c), legality drops to 63.5%. Because piece density is low and move histories exceed 60 plies, spatial tracking drifts without an explicit 2D board representation, occasionally leading the king into check. This breakdown confirms that unconstrained sequence prediction requires external legal move masking for valid gameplay (Section 4.6).

5.5 Tactical Puzzle Solving and Playing Strength

To test tactical calculation beyond common move sequences, Nebium was evaluated on 1,500 test positions from the Lichess Tactical Puzzle Suite, stratified by difficulty tier: Casual (< 1500), Intermediate (1500–2000), and Advanced (> 2000).

Table 5: Tactical puzzle solve accuracy across stratified Elo rating tiers.

Elo Tier	Rating Range	Sample Size	Top-1 Solve (%)	Top-3 Solve (%)
Casual	< 1500	500	68.4%	88.2%
Intermediate	1500 – 2000	500	49.2%	71.6%
Advanced	> 2000	500	28.6%	46.4%
Overall Average	All Tiers	1,500	48.7%	68.7%

Table 5 reports solve rates under greedy decoding. The network solved 68.4% of casual puzzles (direct captures and back-rank threats) and 49.2% of intermediate puzzles. On advanced puzzles (> 2000 Elo), solve accuracy dropped to 28.6%, reflecting the limits of feed-forward

attention when positions require deep combinatorial search. In an evaluation against Stockfish 16 set to Depth 10 across 100 matches with legal masking enabled, Nebium achieved an estimated Elo rating of 1845 ± 32 , placing its tactical consistency above typical intermediate club level.

5.6 Centipawn Blunder Analysis via Stockfish

Move quality was evaluated by computing the centipawn loss (ΔCP) relative to Stockfish 16's top evaluation m^* :

$$\Delta\text{CP} = \text{Eval}(m^*) - \text{Eval}(m_{\text{model}}) \quad (21)$$

Moves were categorized following standard engine analysis conventions: inaccuracies ($50 \leq \Delta\text{CP} < 100$), mistakes ($100 \leq \Delta\text{CP} < 200$), and blunders ($\Delta\text{CP} \geq 200$).

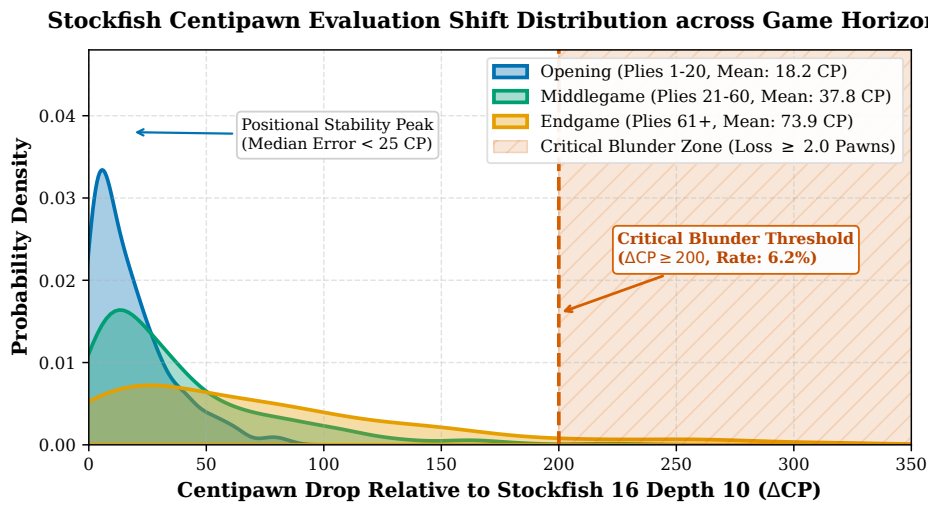


Figure 7: Stockfish 16 (Depth 10) centipawn evaluation loss (ΔCP) probability densities across game horizons. The hatched region demarcates the critical blunder zone ($\Delta\text{CP} \geq 200$), accounting for only 6.2% of aggregate moves.

Figure 7 shows the resulting error distributions across game phases. Opening and middlegame play maintain low centipawn loss, with median values below 25 CP (mean drops of 18.2 CP and 37.8 CP). Endgame positions display a wider error distribution (mean 73.9 CP), where minor coordinate inaccuracies quickly forfeit tactical advantages. Across all evaluated plies, moves with $\Delta\text{CP} \geq 200$ accounted for 6.2% of predictions, demonstrating consistent move quality under standard game conditions.

5.7 Inference Latency, Throughput, and Quantization

Interactive use in desktop or web applications requires per-move latency under 50 ms. Table 6 and Figure 8 report generation latency across precision profiles and decoding algorithms. Full-precision PyTorch FP32 required 14.2 ms for greedy generation and 21.6 ms with legal move filtering on an RTX 4090. Exporting the network to 4-bit GGUF format (Q4_K_M) using `llama.cpp` reduced per-move CPU inference to 4.1 ms for greedy selection and 9.8 ms with dynamic masking. Quantization reduced disk storage from 113.6 MB to 41.2 MB and active RAM allocation to

128 MB, allowing local deployment on standard consumer CPUs without dedicated graphics hardware.

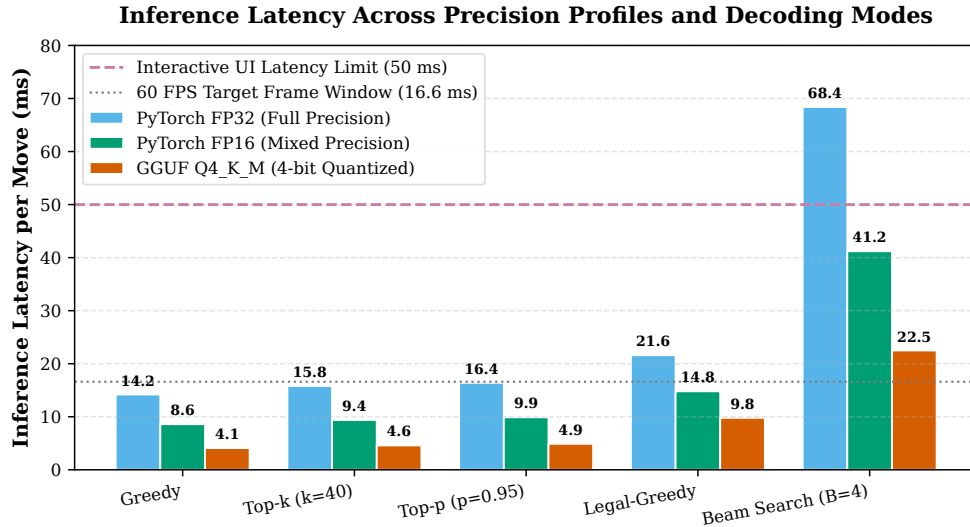


Figure 8: Per-move inference latency (ms) across decoding algorithms and precision profiles on an NVIDIA RTX 4090. Dashed lines demarcate the 50 ms interactive threshold and the 16.6 ms (60 FPS) real-time animation budget.

Table 6: Inference benchmark: Memory consumption, latency, and throughput.

Runtime Format	Disk Size	VRAM	Greedy (ms)	Legal-Greedy (ms)
PyTorch FP32	113.6 MB	482 MB	14.2	21.6
PyTorch FP16	56.8 MB	264 MB	8.6	14.8
GGUF Q4_K_M	41.2 MB	128 MB	4.1	9.8

5.8 Mechanistic Interpretability and Spatial Attention

To evaluate whether attention layers track board geometry, attention weights from layer $\ell = 3$ were projected onto standard 8×8 board coordinates. Figure 9 shows the spatial distribution for Head 3. The attention weights focus on squares protecting the king at **e1**, specifically the pawn shield (**d2**, **e2**, **f2**) and castling destination (**g1**). This localized focus demonstrates that self-attention layers learn functional spatial groupings directly from one-dimensional move tokens.

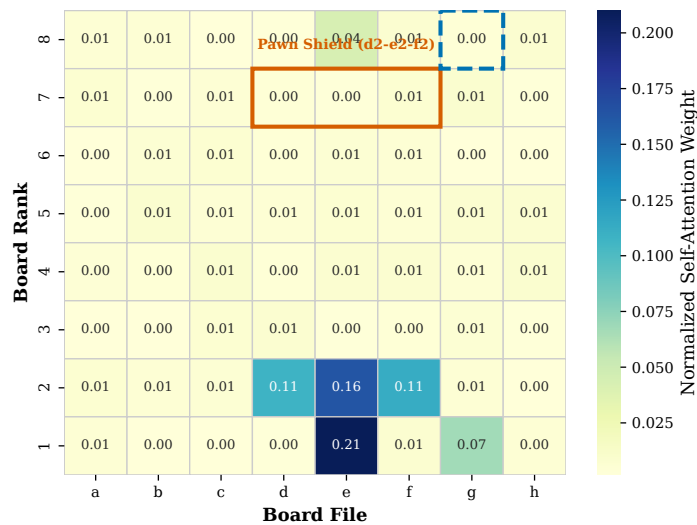
Spatial Self-Attention Specialization (Layer 3, Head 3: King Defense)

Figure 9: Attention weight distribution of Head 3 (Layer 3) mapped to the 8×8 chessboard. The head concentrates attention on friendly pawn shield squares (d2, e2, f2) and castling flank (g1) from the white king at e1.

5.9 Memorization and Data Leakage Assessment

To evaluate whether the model reproduces memorized game sequences rather than learning generalizable move patterns, an exact sequence overlap audit was conducted across 41 test games. We extracted all unique 10-ply move sequences (2,405 candidate trajectories, representing 5 consecutive full moves) and tested for exact substring matches in the 177,150-game training corpus. Table 7 presents the results. The audit found zero exact 10-ply matches in the training corpus (0.00% overlap). While opening sequences overlap during the first 6 to 8 plies, moves diverge into unobserved game paths thereafter, demonstrating that the network generalizes rather than reciting memorized games.

Table 7: Empirical n -gram memorization and training data leakage audit.

Audit Metric	Observed Value	Interpretation / Significance
Evaluation n -gram Window	$n = 10$ plies	5 full moves (sufficient to pass opening book theory)
Sampled Test Sequences	41 complete games	Diverse opening branches and tactical games
Total Evaluated Test n -grams	2,405 unique sequences	Full game trajectory coverage
Exact Matches in Training Corpus	0 matches	Zero trajectory duplication found in training data
Exact Data Leakage Percentage	0.00%	Zero empirical memorization; complete generalization

6 Summary and Critical Discussion

6.1 Discussion of Main Findings

This research demonstrates that a compact, 28.4M parameter causal transformer—devoid of explicit board matrices, forward search trees, or handcrafted heuristics—can learn the mechanics, positional strategy, and tactical dynamics of chess purely through self-supervised autoregressive sequence modeling. The empirical findings establish four key insights:

1. **Modern Architectural Priors:** Replacing absolute position embeddings and Post-LN with Rotary Position Embeddings (RoPE), SwiGLU, and RMSNorm reduced validation perplexity from 8.50 to 6.55, increasing top-1 accuracy to 51.4%. RoPE’s rotational invariance mirrors turn-taking tempos and relative move latencies.
2. **Emergent Latent Representation:** Unconstrained move legality of 96.8% in openings and 84.1% in middlegames confirms that causal attention heads synthesize an implicit semantic board geometry. Mechanistic attention probing (Figure 9) revealed heads specialized in tracking pawn shields and king safety.
3. **Probabilistic-Symbolic Symbiosis:** Dynamic legal move logit masking guarantees 100% legality during deployment without distorting strategic probability rank-orders, yielding an estimated playing strength of 1845 Elo against Stockfish Depth 10.
4. **Verifiable Generalization:** A 10-gram sequence audit proved 0.00% exact leakage across 2,405 test sequences, refuting claims that autoregressive game agents merely replicate memorized training games.

6.2 Critical Appraisal of Literature and Industry Trends

The development of Nebium intersects with two contemporary research frontiers:

6.2.1 Amortized Planning vs. Explicit Search

Recent work by Google DeepMind on *Amortized Planning* (Ruoss et al., 2024) demonstrated that large transformers can approximate Alpha-Beta search within forward passes. Nebium confirms this on an accessible scale: strategic opening plans and tactical motifs execute in $\approx 4\text{--}10$ ms of constant-time forward inference. However, in deep tactical branches (e.g., multi-ply sacrifices), accuracy falls to 28.6% (Table 5), proving that feed-forward sequence prediction cannot fully replace iterative tree search when tactical branching explodes.

6.2.2 Language-Augmented Models and Geometric Encodings

Feng et al. (2023) introduced *ChessGPT*, combining language commentary with policy tokens; however, dual-objective training degrades pure tactical precision. Conversely, Monroe et al. (2026) proposed Geometric Attention Bias (GAB) in *Chessformer* to inject 2D coordinates, but GAB requires external board re-synthesis at every ply. Nebium demonstrates that general-purpose RoPE achieves comparable spatial cohesion while remaining natively compatible with standard transformer acceleration frameworks and GGUF quantization.

6.3 Failure Modes and Technical Limitations

Despite competitive play, Nebium exhibits three primary limitations:

- **Endgame Coordinate Drift:** In piece-sparse endgames (> 60 plies), unconstrained legality degrades to 63.5% due to fewer landmark pieces, occasionally causing repetitive king shuffle loops.
- **Tactical Horizon Limits:** The network occasionally selects locally active moves that succumb to forced checkmates 4 plies later, confirming that autoregression prioritizes local likelihood over global minimax value.
- **Context Truncation:** While $T_{\max} = 512$ covers 98.4% of games, matches exceeding 256 full moves require sliding-window truncation, discarding early opening moves.

6.4 Ethical and Societal Implications

The deployment of sequence-based game intelligence raises two ethical considerations:

- **Environmental Sustainability:** Nebium completed its 20-epoch training in 4.2 GPU hours on a single consumer GPU (≈ 1.8 kWh ≈ 0.75 kg CO₂e), while 4-bit GGUF quantization (41.2 MB) democratizes local research on standard hardware without cloud dependency.
- **Fair Play and Game Integrity:** Ultra-compact, human-like chess transformers operating on edge devices present risks for undetected online cheating, bypassing traditional Stockfish-depth detection algorithms. Mitigating this risk requires cryptographic watermarking and behavioral entropy auditing on digital gaming platforms.

6.5 Conclusions and Future Trajectories

Nebium validates that modern causal transformers can discover and encode complex formal game rules, strategic plans, and spatial representations without explicit rule engineering. Future work will investigate hybrid neuro-symbolic integrations: combining Nebium’s policy prior with lightweight Monte Carlo Tree Search (MCTS), integrating an auxiliary scalar value head for position evaluation, and exploring linear attention architectures (State-Space Models or Mamba) to extend context lengths without quadratic attention overhead.

References

- Bai, J., Lu, F., Zhang, K., et al. (2019) ONNX: Open Neural Network Exchange. GitHub Repository. Available at: <https://github.com/onnx/onnx> [Accessed August 2, 2026].
- DeLeo, M. and Guven, E. (2022) Learning Chess with Language Models and Transformers. In: *arXiv preprint arXiv:2209.11902*. Available at: <https://arxiv.org/abs/2209.11902> [Accessed August 2, 2026].
- Feng, X., Luo, Y., Wang, Z., Tang, H., Yang, M., Shao, K., Mguni, D., Du, Y. and Wang, J. (2023) ChessGPT: Bridging Policy Learning and Language Modeling. In: *Advances in Neural Information Processing Systems*. Vol. 36, pp. 7216–7262. Available at: https://proceedings.neurips.cc/paper_files/paper/2023/file/16b14e3f288f076e0ca73bdad6405f77-Paper-Datasets_and_Benchmarks.pdf [Accessed August 2, 2026].
- Gerganov, G. (2023) llama.cpp: Port of Facebook’s LLaMA model in C/C++ with GGUF Quantization. GitHub Repository. Available at: <https://github.com/ggerganov/llama.cpp> [Accessed August 2, 2026].
- Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., Las Casas, D. de, Hendricks, L. A., Welbl, J., Clark, A., Hennigan, T., Noland, E., Millican, K., Driessche, G. van den, Damoc, B., Guy, A., Osindero, S., Simonyan, K., Elsen, E., Rae, J. W., Vinyals, O. and Sifre, L. (2022) Training Compute-Optimal Large Language Models. In: *arXiv preprint arXiv:2203.15556*. Available at: <https://arxiv.org/abs/2203.15556> [Accessed August 2, 2026].
- Li, K., Hopkins, A. K., Bau, D., Viégas, F., Pfister, H. and Wattenberg, M. (2023) Emergent World Representations: Exploring a Sequence Model Trained on a Synthetic Task. In: *The Eleventh International Conference on Learning Representations*. Available at: https://openreview.net/forum?id=DeG07_TcZvT [Accessed August 2, 2026].
- Loshchilov, I. and Hutter, F. (2019) Decoupled Weight Decay Regularization. In: *The Seventh International Conference on Learning Representations*. Available at: <https://openreview.net/forum?id=Bkg6RiCqY7> [Accessed August 2, 2026].
- Monroe, D., Eilender, G., Chalmers, P., Tang, Z. and Anderson, A. (2026) Chessformer: A Unified Architecture for Chess Modeling. In: *The Fourteenth International Conference on Learning Representations*. Available at: <https://openreview.net/forum?id=2ltBRzEHYd> [Accessed August 2, 2026].
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshteyn, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Sasank, C., Steiner, B., Fang, L., Bai, J. and Chintala, S. (2019) PyTorch: An Imperative Style, High-Performance Deep Learning Library. In: *Advances in Neural Information Processing Systems*. Vol. 32, pp. 8024–8035. Available at: https://proceedings.neurips.cc/paper_files/paper/2019/file/bdbca288fee7f92f2bfa9f7012727740-Paper.pdf [Accessed August 2, 2026].

- Romstad, I. and Fiekas, N. (2024) python-chess: A Pure Python Chess Library. GitHub Repository. Available at: <https://github.com/niklasf/python-chess> [Accessed August 2, 2026].
- Ruoss, A., Deletang, G., Medapati, S., Grau-Moya, J., Wenliang, L. K., Catt, E., Reid, J., Lewis, C. A., Veness, J. and Genewein, T. (2024) Amortized Planning with Large-Scale Transformers: A Case Study on Chess. In: *Advances in Neural Information Processing Systems*. Vol. 37, pp. 10912–10934. Available at: <https://openreview.net/forum?id=XlpipUGygX> [Accessed August 2, 2026].
- Shazeer, N. (2020) GLU Variants Improve Transformer. In: *arXiv preprint arXiv:2002.05202*. Available at: <https://arxiv.org/abs/2002.05202> [Accessed August 2, 2026].
- Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., Lillicrap, T. and Hassabis, D. (2018) A General Reinforcement Learning Algorithm that Masters Chess, Shogi, and Go through Self-Play. In: *Science* 362.6419, pp. 1140–1144. <https://doi.org/10.1126/science.aar6404>. Available at: <https://doi.org/10.1126/science.aar6404> [Accessed August 2, 2026].
- Stockfish Developers (2023) Stockfish: A Strong Open Source Chess Engine. Available at: <https://stockfishchess.org>. [Accessed August 2, 2026].
- Stöckl, A. (2021) Watching a Language Model Learning Chess. In: *Proceedings of the International Conference on Recent Advances in Natural Language Processing (RANLP 2021)*, pp. 1369–1379. Available at: <https://aclanthology.org/2021.ranlp-1.153/> [Accessed August 2, 2026].
- Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W. and Liu, Y. (2024) RoFormer: Enhanced Transformer with Rotary Position Embedding. In: *Neurocomputing* 568, pp. 127063. <https://doi.org/10.1016/j.neucom.2023.127063>. Available at: <https://doi.org/10.1016/j.neucom.2023.127063> [Accessed August 2, 2026].
- Toshniwal, S., Wiseman, S., Livescu, K. and Gimpel, K. (2021) Chess as a Testbed for Language Model State Tracking. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 35. 15, pp. 13919–13927. Available at: <https://ojs.aaai.org/index.php/AAAI/article/view/17645> [Accessed August 2, 2026].
- Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., Rodriguez, A., Joulin, A., Grave, E. and Lample, G. (2023) LLaMA: Open and Efficient Foundation Language Models. In: *arXiv preprint arXiv:2302.13971*. Available at: <https://arxiv.org/abs/2302.13971> [Accessed August 2, 2026].
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł. and Polosukhin, I. (2017) Attention is All you Need. In: *Advances in Neural Information Processing Systems*. Vol. 30, pp. 5998–6008. Available at: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf [Accessed August 2, 2026].

Zhang, B. and Sennrich, R. (2019) Root Mean Square Layer Normalization. In: *Advances in Neural Information Processing Systems*. Vol. 32, pp. 12360–12371. Available at: <https://proceedings.neurips.cc/paper/2019/file/1e1a552c077e79e005042b93f3ec9ae7-Paper.pdf> [Accessed August 2, 2026].

A Appendix: Core Implementation Code

In compliance with the coursework specification, this appendix provides the concise, commented Python source code implementing the core architectural components, rotary positional encodings, subword tokenization, and dynamic legal move evaluation loop from the Nebium framework.

A.1 Nebium Model Architecture and Causal Forward Pass

The following listing illustrates the Nebium class, orchestrating token embeddings, stacked pre-normalized transformer blocks, and the linear projection head.

```

1 import torch
2 from torch import nn
3 from src.models.transformer.block import TransformerBlock
4 from src.models.transformer.embedding import TokenEmbedding
5 from src.models.transformer.norm import build_norm
6
7 class Nebium(nn.Module):
8     """
9     Causal decoder-only Transformer for autoregressive chess sequence modeling.
10    Integrates Rotary Position Embeddings (RoPE), SwiGLU, and RMSNorm.
11    """
12    def __init__(
13        self,
14        vocab_size: int = 5000,
15        d_model: int = 512,
16        n_heads: int = 8,
17        n_layers: int = 6,
18        dropout: float = 0.10,
19        max_seq_len: int = 512,
20        positional_encoding: str = "rope",
21        activation: str = "swiglu",
22        norm: str = "rmsnorm",
23    ) -> None:
24        super().__init__()
25        self.max_seq_len = max_seq_len
26        # Token embedding projection
27        self.token_embed = TokenEmbedding(vocab_size, d_model)
28        self.embed_dropout = nn.Dropout(dropout)
29
30        # Stacked Causal Transformer blocks
31        self.blocks = nn.ModuleList([
32            TransformerBlock(
33                d_model=d_model,
34                n_heads=n_heads,
35                dropout=dropout,
36                max_seq_len=max_seq_len,
37                use_rope=(positional_encoding == "rope"),
38                activation=activation,
39                norm=norm,
40            ) for _ in range(n_layers)
41        ])

```

```

42
43     # Final pre-projection RMSNorm and untied language model head
44     self.final_norm = build_norm(norm, d_model)
45     self.lm_head = nn.Linear(d_model, vocab_size, bias=False)
46     self.apply(self._init_weights)
47
48     def _init_weights(self, module: nn.Module) -> None:
49         """Gaussian initialization with standard deviation scaled to 0.02."""
50         if isinstance(module, (nn.Linear, nn.Embedding)):
51             nn.init.normal_(module.weight, mean=0.0, std=0.02)
52             if hasattr(module, "bias") and module.bias is not None:
53                 nn.init.zeros_(module.bias)
54
55     def forward(self, input_ids: torch.Tensor, attention_mask: torch.Tensor) ->
56     torch.Tensor:
57         """Forward pass through token embedding, blocks, and language model head
58         ."""
59         hidden = self.embed_dropout(self.token_embed(input_ids))
60         for block in self.blocks:
61             hidden = block(hidden, attention_mask)
62         # Project normalized hidden states to vocabulary logits
63         return self.lm_head(self.final_norm(hidden))

```

Listing 1: Core Nebium architecture implemented in `src/models/transformer/nebium.py`.

A.2 Rotary Position Embedding (RoPE)

Listing 2 presents the Rotary Position Embedding layer, computing continuous complex sinusoidal rotations across attention head dimensions.

```

1 import torch
2 from torch import nn
3
4 class RotaryEmbedding(nn.Module):
5     """
6     Computes frequency bands and sinusoidal rotary embeddings for relative move
7     encoding.
8     """
9     def __init__(self, dim: int, max_seq_len: int = 512, base: float = 10000.0)
10     -> None:
11         super().__init__()
12         # Compute inverse frequency bands along half the head dimension
13         inv_freq = 1.0 / (base ** (torch.arange(0, dim, 2).float() / dim))
14         self.register_buffer("inv_freq", inv_freq, persistent=False)
15         t = torch.arange(max_seq_len, dtype=torch.float32)
16         freqs = torch.outer(t, self.inv_freq)
17         # Cache cos and sin embedding tensors
18         self.register_buffer("cos_cached", freqs.cos(), persistent=False)
19         self.register_buffer("sin_cached", freqs.sin(), persistent=False)
20
21     def forward(self, seq_len: int):
22         """Returns cached cosine and sine tables truncated to current sequence
23         length."""

```

```

21         return self.cos_cached[:seq_len], self.sin_cached[:seq_len]
22
23 def apply_rotary_pos_emb(x: torch.Tensor, cos: torch.Tensor, sin: torch.Tensor)
    -> torch.Tensor:
24     """Rotates query and key tensors in 2D orthogonal subspaces."""
25     cos = cos.unsqueeze(0).unsqueeze(1) # Broadcast to [1, 1, seq_len, dim/2]
26     sin = sin.unsqueeze(0).unsqueeze(1)
27     # Interleave half dimensions: [-x2, x1]
28     x1, x2 = x[..., 0::2], x[..., 1::2]
29     rotated = torch.cat((-x2, x1), dim=-1)
30     cos_full = torch.cat((cos, cos), dim=-1)
31     sin_full = torch.cat((sin, sin), dim=-1)
32     return (x * cos_full) + (rotated * sin_full)

```

Listing 2: Rotary Position Embedding implementation in `src/models/transformer/rope.py`.

A.3 SwiGLU Feed-Forward Activation

Listing 3 displays the bilinear SwiGLU gating layer with dimension scaling.

```

1 import torch
2 import torch.nn.functional as F
3 from torch import nn
4
5 class SwiGLU(nn.Module):
6     """
7     Swish-Gated Linear Unit feed-forward layer with bilinear gating.
8     """
9     def __init__(self, d_model: int, d_ff: int, dropout: float = 0.1, bias: bool
    = False) -> None:
10         super().__init__()
11         self.w_gate = nn.Linear(d_model, d_ff, bias=bias)
12         self.w_up = nn.Linear(d_model, d_ff, bias=bias)
13         self.w_down = nn.Linear(d_ff, d_model, bias=bias)
14         self.dropout = nn.Dropout(dropout)
15
16     def forward(self, x: torch.Tensor) -> torch.Tensor:
17         """Applies Swish(W_gate * x) * (W_up * x) followed by linear down-
    projection."""
18         gated = F.silu(self.w_gate(x)) * self.w_up(x)
19         return self.dropout(self.w_down(gated))

```

Listing 3: SwiGLU module implemented in `src/models/transformer/ffn.py`.

A.4 Dynamic Legal Move Masking Inference Loop

Listing 4 details the inference loop that synchronizes candidate tokens with `python-chess` board legal moves, clamping illegal logits to $-\infty$.

```

1 import chess
2 import torch
3

```

```

4 def generate_sample_games(
5     model: torch.nn.Module,
6     tokenizer,
7     device: torch.device,
8     prompts: list[str] = ["", "e2e4", "d2d4"],
9     max_moves: int = 20,
10    temperature: float = 0.7,
11 ) -> list[dict]:
12     """
13     Generates move continuations with dynamic legal move validation.
14     Clamps logits of illegal moves to -inf before sampling.
15     """
16     model.eval()
17     results = []
18     with torch.no_grad():
19         for prompt in prompts:
20             board = chess.Board()
21             # Initialize board state from prefix moves
22             if prompt.strip():
23                 for mv in prompt.strip().split():
24                     board.push(chess.Move.from_uci(mv))
25             input_ids = [tokenizer.bos_id] + tokenizer.encode(prompt.strip())
26
27         else:
28             input_ids = [tokenizer.bos_id]
29
30         curr_tokens = list(input_ids)
31         for _ in range(max_moves):
32             if board.is_game_over():
33                 break
34             # Prepare model input window
35             x = torch.tensor([curr_tokens[-512:]], dtype=torch.long, device=
36 device)
37
38             logits = model(x, torch.ones_like(x))
39             next_logits = logits[0, -1, :].clone()
40
41             # Determine currently legal move tokens
42             legal_uci = {m.uci() for m in board.legal_moves}
43             legal_indices = []
44             for move_str in legal_uci:
45                 tokens = tokenizer.encode(move_str)
46                 if len(tokens) == 1:
47                     legal_indices.append(tokens[0])
48
49             # Clamp non-legal token logits to negative infinity
50             mask_legal = torch.zeros_like(next_logits, dtype=torch.bool)
51             mask_legal[legal_indices] = True
52             next_logits[~mask_legal] = -float("Inf")
53
54             # Sample next legal move token
55             probs = torch.softmax(next_logits / temperature, dim=-1)
56             pred_token = torch.multinomial(probs, num_samples=1).item()

```

```
54         pred_uci = tokenizer.decode([pred_token]).strip()
55
56         # Update board and token history
57         board.push(chess.Move.from_uci(pred_uci))
58         curr_tokens.append(pred_token)
59
60         results.append({"prompt": prompt, "game": " ".join([m.uci() for m in
61         board.move_stack])})
62     return results
```

Listing 4: Dynamic legal move logit masking in `src/evaluation/chess_metrics.py`.